

Policy Gradient

Zhe Gao

School of Management
University of Science and Technology of China

June 4, 2025

Outline

- 1 Introduction
- 2 Policy Gradient Methods

Reinforcement Learning

What is RL?

- Learning paradigm where an **agent** learns optimal behaviors through **trial-and-error** interactions
- Receives evaluative feedback (**rewards**) instead of direct instructions
- Core challenge: **Exploration vs Exploitation** dilemma

Key Characteristics:

- **Sequential decision making:** Actions have long-term consequences
- **Delayed rewards:** Consequences may appear many steps later
- **Online learning:** Learns by interacting with environment

Reinforcement Learning

Diverse Real-World Applications:

- **Robotics:** Autonomous navigation, manipulation tasks
- **Game AI:** AlphaGo, Dota 2 bots, StarCraft II agents
- **Recommendation Systems:** Personalized content delivery
- **Finance:** Optimal trading strategies
- **Healthcare:** Treatment policy optimization

Reinforcement Learning

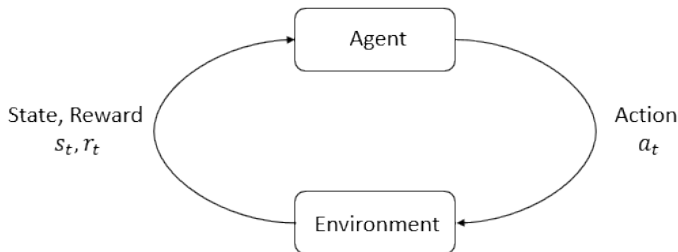


Figure: RL framework

Interaction Loop:

- ① Agent observes state S_t
- ② Selects action $A_t \sim \pi(\cdot | S_t)$
- ③ Receives reward R_{t+1}
- ④ Transitions to new state S_{t+1}

Reinforcement Learning

Key Components:

S_t : State (environment observation)

A_t : Action (agent's decision)

R_{t+1} : Reward (performance feedback)

Goal: maximize the (expected) return:

$$\max_{\pi} E \left[\sum_{t=0}^{\infty} \gamma^t R_t \right]$$

- π : Policy (decision strategy)
- γ : Discount factor ($0 \leq \gamma < 1$)

Markov Decision Process (MDP)

Markov Property:

$$\Pr(S_{t+1}|S_t, A_t) = \Pr(S_{t+1}|S_t, A_t, \text{history})$$

"The future is independent of the past given the present"

Returns and Value Functions

Cumulative Return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

State-Value Function:

$$V^{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

"Expected return starting from state s , following policy π "

Action-Value Function:

$$Q^{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

"Expected return after taking action a in state s , then following π "

Connection:

$$V^{\pi}(s) = \sum_a \pi(a|s) Q^{\pi}(s, a)$$

Bellman Equations

Fundamental Recursive Relationships

Bellman Equation for V^π :

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [r(s, a) + \gamma V^\pi(s')]$$

Bellman Optimality Equation:

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \right]$$

Key Insight:

- Value functions satisfy recursive relationships
- Enables dynamic programming solutions
- Foundation for most RL algorithms

Two main approaches

There are two approaches to train our agent to find this optimal policy π^* :

- Teach the agent to learn which action to take, given the current state: **Policy-Based Methods**.
- Teach the agent to learn which state is more valuable: **Value-Based Methods**.

Value-Based Methods

- In value-based methods, we focus on learning a value function, usually the value function of a state-action pair $Q(s, a)$, which represents the expected reward of taking action a in state s .
- The policy is usually derived from this value function, such as the most common greedy policy, which is to choose the action that maximizes $Q(s, a)$ in each state.
- The policy is usually deterministic because it always chooses the action that seems to be the best at the moment.

Core Idea: Learn value functions $\rightarrow$ Derive policy

$$\pi(s) = \arg \max_a Q(s, a)$$

Value-Based Methods

Representative Algorithms:

- Q-Learning: $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
- DQN (Deep Q-Networks): Combines Q-learning with deep neural networks
- Value Iteration: $V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [r(s, a) + \gamma V_k(s')]$

Value-Based Methods

Strengths:

- Sample efficient (reuses experience)
- Stable convergence in tabular cases
- Clear theoretical foundations

Limitations:

- Curse of dimensionality: Action space explosion in continuous domains
- Deterministic policies: Cannot naturally represent stochastic optimal policies
- Discontinuous policies: Small value changes cause abrupt policy shifts
- Indirect optimization: Policy derived from value estimates

Policy-Based Methods

- Directly parameterize and optimize the policy $\pi(a | s)$, which directly outputs the probability of taking each possible action in a given state.
- It can be naturally represented as a stochastic policy, which is very useful in some environments, such as those involving incomplete information or multiple equally good actions.

Direct Policy Optimization:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

where $J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right]$ (Expected value of long-term returns)

Policy-Based Methods

Representation Flexibility

- Handles **continuous action spaces**
- Naturally represents **stochastic policies**
- Accommodates **structured policies**

Optimization Properties

- **Smoother convergence**
- Better **local optimal** guarantees
- Effective in **partially observable** environments

Policy-Based Methods

Property	Value-Based	Policy-Based
Action space	Discrete	Continuous
Policy type	Deterministic	Stochastic
Convergence	Oscillatory	Stable

Outline

1 Introduction

2 Policy Gradient Methods

- Proximal Policy Optimization Algorithms (PPO)
- Deep Deterministic Policy Gradient

Policy Gradient Core Idea

Key Insight: Estimate gradient of expected return

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

where $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ (return from time t)

Fundamental Equation

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

Policy Gradient Core Idea

Derivation Steps:

- ① Start with objective: $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$
- ② Gradient: $\nabla_\theta J(\theta) = \nabla_\theta \int P(\tau; \theta) R(\tau) d\tau$
- ③ Likelihood ratio trick: $\nabla_\theta P(\tau; \theta) = P(\tau; \theta) \nabla_\theta \log P(\tau; \theta)$
- ④ Decompose trajectory probability:

$$P(\tau; \theta) = p(s_0) \prod_{t=0}^T \pi_\theta(a_t | s_t) P(s_{t+1} | s_t, a_t)$$
- ⑤ Log-derivative: $\nabla_\theta \log P(\tau; \theta) = \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t)$

Intuition:

- **Increase** probability of actions that led to **high return**
- **Decrease** probability of actions that led to **low return**
- Update magnitude proportional to G_t

Outline

1 Introduction

2 Policy Gradient Methods

- Proximal Policy Optimization Algorithms (PPO)
- Deep Deterministic Policy Gradient

Motivation

- Q-learning (with function approximation) fails on many simple problems and is poorly understood;
- Vanilla policy gradient methods have poor data efficiency and robustness;
- Trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

To optimize policies, we alternate between sampling data from the policy and performing several epochs of optimization on the sampled data.

Policy Gradient Methods

- Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm,

$$\hat{g} = \hat{\mathbb{E}}_t [\nabla_{\theta} \log \pi_{\theta} (a_t | s_t) \hat{A}_t]$$

where π_{θ} is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t .

- Here, the expectation $\hat{\mathbb{E}}_t[\dots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization.
- The estimator $\hat{g}$ is obtained by differentiating the objective

$$L^G(\theta) = \hat{\mathbb{E}}_t [\log \pi_{\theta} (a_t | s_t) \hat{A}_t]$$

Trust Region Methods

- In TRPO, an objective function is maximized subject to a constraint on the size of the policy update. Specifically,

$$\begin{array}{ll} \underset{\theta}{\text{maximize}} & \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right] \\ \text{subject to} & \hat{\mathbb{E}}_t \left[\text{KL} \left[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t) \right] \right] \leq \delta \end{array}$$

- This problem can efficiently be approximately solved using the **conjugate gradient algorithm**, after making a linear approximation to the objective and a quadratic approximation to the constraint.

Trust Region Methods

- The theory justifying TRPO actually suggests using a penalty instead of a constraint, i.e., solving the unconstrained optimization problem

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL} [\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

for some coefficient β .

- TRPO uses a hard constraint rather than a penalty because it is hard to choose a **single value of** β that performs well across different problems.
- It is not sufficient to simply choose a fixed penalty coefficient β and optimize the penalized objective Equation with SGD; additional modifications are required.

PPO-Clip

- Let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, the PPO-Clip variant introduces a clipping mechanism to the objective function:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

- ϵ is typically set between 0.1 and 0.2.
- The second term modifies the surrogate objective by clipping the probability ratio, which removes the incentive for moving $r_t(\theta)$ outside of the interval $[1 - \epsilon, 1 + \epsilon]$.
- With this scheme, we only ignore the change in probability ratio when it would make the objective improve, and we include it when it makes the objective worse.

Clip function

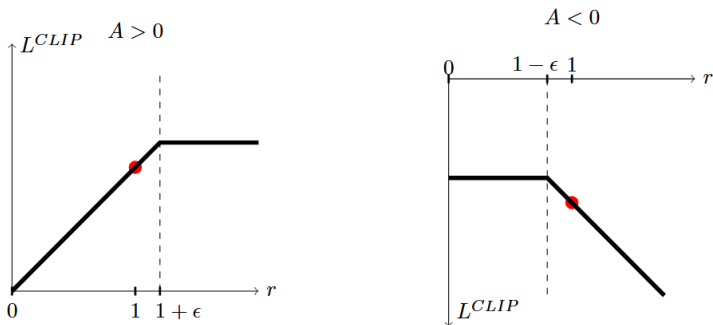


Figure: Clip function

PPO with Penalty

- Using several epochs of minibatch SGD, optimize the KL-penalized objective:

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL} [\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

- Compute $d = \hat{\mathbb{E}}_t [\text{KL} [\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$
 - If $d < d_{\text{targ}}/1.5, \beta \leftarrow \beta/2$
 - If $d > d_{\text{targ}} \times 1.5, \beta \leftarrow \beta \times 2$
- The initial value of β is another hyperparameter but is not important in practice because the algorithm quickly adjusts it.

Algorithm

A proximal policy optimization (PPO) algorithm that uses fixed-length trajectory segments is shown below. Each iteration, each of N (parallel) actors collect T timesteps of data. Then we construct the surrogate loss on these NT timesteps of data, and optimize it with minibatch SGD (or usually for better performance, Adam), for K epochs.

Algorithm 1 PPO, Actor-Critic Style

```
for iteration=1, 2, ... do
  for actor=1, 2, ..., N do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for
```

Outline

1 Introduction

2 Policy Gradient Methods

- Proximal Policy Optimization Algorithms (PPO)
- Deep Deterministic Policy Gradient

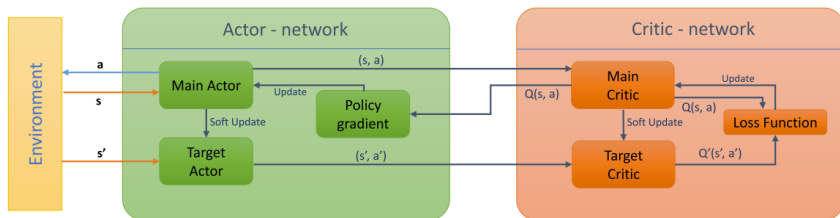
Background

- The use of large, non-linear function approximators for learning value or action-value functions has often been avoided in the past since theoretical performance guarantees are impossible, and practically learning tends to be unstable.
- It is not possible to straightforwardly apply Q-learning to continuous action spaces, because in continuous spaces finding the greedy policy requires an optimization of at every timestep; this optimization is too slow to be practical with large, unconstrained function approximators and nontrivial action spaces.
- Given that the action spaces are continuous, the function $Q(s, a)$ is expected to be differentiable with respect to the action argument. This enables the establishment of an efficient, gradient-based learning rule for a policy.

Overview

- DDPG is a model-free, off-policy RL algorithm that adopts an actor-critic approach to solving continuous control problems in which the action space is continuous. (Q-learning and actor-critic algorithms)
- It memorizes the Q-function utilizing off-policy data point, the Bellman equation, and then employs the Q-function to learn the policy. This method is precisely related to Q-learning and is defined similarly as the most desirable action-value function.

Overview



DDPG-Q-learning sides

- The Bellman-based formula explaining the optimal actions and value functions, $Q^*(s, a)$ is presented via the following equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P} [r(s, a) + \gamma \max_{a'} Q^*(s', a')]$$

- The Bellman-based formula is the starting point for developing an approximate function for $Q^*(s, a)$, the approximator is a neural network $Q^\phi(s, a)$.
- A mean-squared Bellman error (MSBE) function can be constructed to measure how well Q^ϕ satisfies the Bellman-based equation:

$$L(\phi, \mathcal{D}) = \mathbb{E}[(s, a, r, s', d) \sim \mathcal{D}] \left[\left(Q_\phi(s, a) - \left(r + \gamma(1 - d) \max_{a'} Q_\phi(s', a') \right) \right)^2 \right]$$

DDPG-Q-learning sides

- Deep Q-Network (DQN) and all of its variants, as well as DDPG, are Q-learning-based algorithms for function approximators that are primarily based on minimizing MSBE loss functions.
- **Replay Buffer:** The standard algorithms utilized for training a DNN to approximate $Q^*(s, a)$ all rely on an experiences replay buffer, which comprises a set of experiences denoted as $\mathcal{D}$.
- **The target value:** Q-learning algorithm makes utilization of networks that are targeted:

$$r + \gamma(1 - d) \max_{a'} Q_{\phi}(s', a')$$

This function is referred to as the target to reduce the MSBE loss, the Q-function is modeled after it.

DDPG-Q-learning sides

The target value (continue):

- Utilize a set of parameters that approaches, although with a time delay, to respond to a second network, known as the target network, which lags behind the first.

$$\phi_{\text{targ}} \leftarrow \rho \phi_{\text{targ}} + (1 - \rho) \phi$$

- In DQN algorithms, the target-based network is replicated from the original network after a predetermined number of steps. Polyak (ρ) averaging is employed to modernize the target network once every main network update in DDPG-style algorithms.

DDPG-Q-learning sides

The target value (continue):

- Calculating the maximum over actions in the targets is a hard task in continuous action spaces. DDPG deals with this by way of utilizing the target policy network to calculate actions that approximately maximize $Q_{\phi_{\text{targ}}}$.
- The target policy-based network is found to be similar to the target Q-function: it is achieved by taking a polyak average of the policy parameters during training.
- In DDPG, Q-learning is performed by minimizing the MSBE losses using stochastic gradient descent (targets policy).

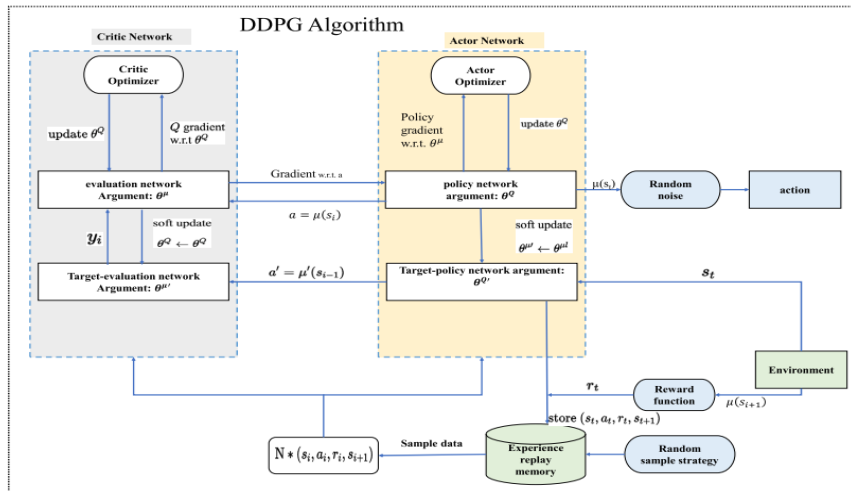
DDPG-policies learning side

- Assuming the Q-functions are differentiable regarding action and that the actions, and spaces are continuous, gradient ascent concerning policy parameters alone may be used to solve the problem.
- The algorithm of DDPG involves two neural networks: The actor and the critic networks and they have distinct roles.

DDPG-policies learning side

- The actor network is in charge of acquiring knowledge about the best policy function, which is used to map states to actions. The policy function is a representation of map states to actions. The actor network receives the current state as input and produces an action as output, which is then transmitted to the environment.
- The critic network has the responsibility of learning the Q-value function, which represents the expected total reward that results from taking a specific action in a particular state and following the policy thereafter. The critic network takes in the current state and action as input and produces the corresponding Q-value as output.
- The DDPG algorithm's learning procedures entail modifying the weights of both the actor network and the critic network via gradient descent.

DDPG-policies learning side

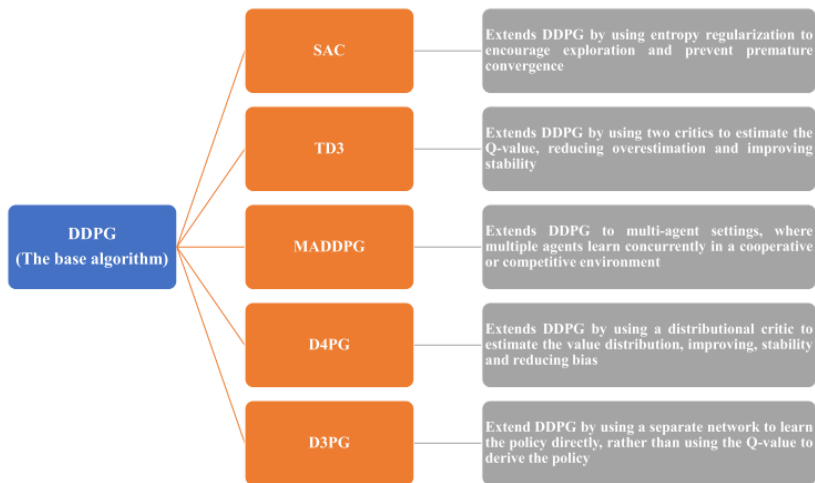


Extensions and modifications of DDPG

DDPG has several limitations, such as the tendency to get stuck in local optima, sensitivity to hyperparameters, and difficulty handling continuous action spaces.

- Improving robustness: DDPG can be sensitive to the choice of hyperparameters and the initialization of the neural networks.
- Handling new challenges: The field of DRL is constantly evolving, with new challenges and applications being proposed.
- Incorporating new ideas: DRL is an interdisciplinary field, with ideas from machine learning, control theory, and artificial intelligence being incorporated into the development of new algorithms.

Extensions and modifications of DDPG



References

- Sutton R S, McAllester D, Singh S, et al. Policy gradient methods for reinforcement learning with function approximation[J]. Advances in neural information processing systems, 1999, 12.
- Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms[J]. arXiv preprint arXiv:1707.06347, 2017.
- Lillicrap T P, Hunt J J, Pritzel A, et al. Continuous control with deep reinforcement learning[J]. arXiv preprint arXiv:1509.02971, 2015.
- Sumiea E H, Abdulkadir S J, Alhussian H S, et al. Deep deterministic policy gradient algorithm: A systematic review[J]. Heliyon, 2024.